

LAB3_P3

May 4, 2026

AI TECH - Akademia Innowacyjnych Zastosowań Technologii Cyfrowych. Programu Operacyjnego Polska Cyfrowa na lata 2014-2020 AI-TECH

Przykład 3 DO LABORATORIUM 3

z UCZENIA MASZYNOWEGO

Klasyfikacja nienadzorowana

Adam Kurowski

Porównanie jakości segmentacji tekstu za pomocą algorytmu k średnich, hierarchicznego i bazującego na gęstości punktów (DBSCAN)

</html>



Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 “Cyfrowe kompetencje społeczeństwa” Działanie nr 3.2 “Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej” Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

1 Przykład 3.: Porównanie jakości segmentacji tekstu za pomocą algorytmu k średnich, hierarchicznego i bazującego na gęstości punktów (DBSCAN)

1.1 Przygotowanie danych wejściowych

W tym przykładzie zajmiemy się badaniem zachowania się algorytmów klasyfikacji nienadzorowanej w przypadku zastosowania tego typu narzędzi do segmentacji pisma. Klasyfikacja nienadzorowana jest w takim przypadku niezbędna, jeżeli nie zakładamy z góry, jakimi znakami posługuje się osoba wykonująca dany zapis. Nie możemy zatem przeprowadzić klasyfikacji nadzorowanej, jednak nadal możemy posłużyć się algorytmem działającym w sposób nienadzorowany do podzielenia poszczególnych obszarów zapisu na pojedyncze znaki.

```
[1]: # Import bibliotek
import numpy as np # obliczenia numeryczne
import pandas as pd # przetwarzanie danych "tabelarycznych"
import matplotlib.pyplot as plt # wizualizacja danych
from sklearn.datasets import make_blobs, make_circles, make_moons # zbiory
    ↪ danych

from sklearn.decomposition import PCA # redukcja wymiarowości
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN #
    ↪ implementacja algorytmów klasyfikacji

#
    ↪ nienadzorowanej

import scipy.cluster.hierarchy as sch # procedury potrzebne do wizualizacji
    ↪ działania algorytmów hierarchicznych

from PIL import Image # biblioteka do wczytywania i przetwarzania obrazów
import matplotlib.cm as cm # dodatkowe palety kolorów - do łatwego ich
    ↪ nadawania efektom klasteryzacji w których

# generuje się bardzo dużo klastrów
```

Na potrzeby przykładu posłużymy się dwoma obrazkami zawierającymi pismo. Pierwszy z nich (logo.png) zawiera odręcznie narysowane logo fikcyjnej firmy. Drugi (wzor_matematyczny.png) zawiera przykład wzoru matematycznego.

```
[2]: # Załadowanie danych z wybranego obrazka zawierającego próbkę pisma:
image = Image.open("logo.png") # logo fikcyjnej firmy
# image = Image.open("wzor_matematyczny.png") # wzór matematyczny

# Wyświetlenie podglądu danych oryginalnych
plt.figure(figsize=(16,9))
plt.imshow(image)
plt.xlabel('x [piksele]', fontsize=15)
plt.ylabel('y [piksele]', fontsize=15)
plt.title('Podgląd wejściowego obrazka zawierającego próbkę pisma', fontsize=15)
```

```
[2]: Text(0.5, 1.0, 'Podgląd wejściowego obrazka zawierającego próbkę pisma')
```



Dane w postaci kolorowej grafiki (pomimo czarno-białej szaty graficznej zawiera ona kanały R,G,B i A) nie nadają się do dalszego przetwarzania. Konieczna jest konwersja grafiki do formatu czarno-białego i odszukanie tych pikseli, które są czarne.

```
[3]: # Konwersja grafiki do formatu czarno-białego (bez szarości).
image_bw = np.array(image.convert('1'))

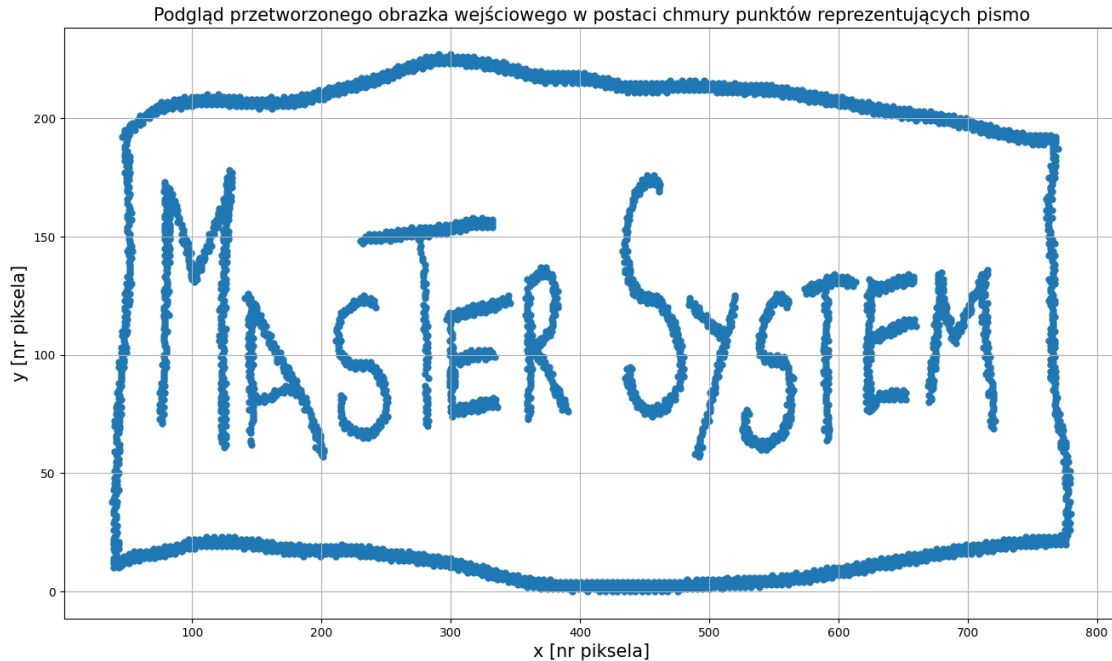
# Odszukanie współrzędnych, dla których piksel jest czarny
coords = np.where(image_bw==0)

# Współrzędne mają w obrazkach w bibliote Pillow inny układ współrzędnych niż
    ↪ ten, z którego potem będziemy
# korzystać w bibliotece scikit-learn, dlatego tutaj musimy zamienić je
    ↪ kolejnościami, oraz musimy "odwrócić"
# oś y.
dataset = np.vstack([coords[1], np.max(coords[0]) - coords[0]]).T

# Dodatkowo zmniejszymy czterokrotnie ilość danych, aby przyspieszyć obliczenia.
dataset = dataset[::4,:]

# Po wykonaniu wszystkich czynności przygotowawczych, wyświetlamy efekt
    ↪ wykonania kodu.
plt.figure(figsize=(16,9))
plt.scatter(dataset[:,0],dataset[:,1],s=25)
plt.grid()
plt.xlabel('x [nr piksela]', fontsize=15)
plt.ylabel('y [nr piksela]', fontsize=15)
plt.title('Podgląd przetworzonego obrazka wejściowego w postaci chmury punktów
    ↪ reprezentujących pismo', fontsize=15)
```

```
[3]: Text(0.5, 1.0, 'Podgląd przetworzonego obrazka wejściowego w postaci chmury
punktów reprezentujących pismo')
```



1.2 Algorytm k średnich

Zadaniem do wykonania w ramach przykładu jest takie przeprowadzenie klasyfikacji nienadzorowanej, aby do każdej klasy trafił pojedynczy znak pisma (ewentualnie fragment znaku, tak jak kropka w literze i). Na początek zbadamy, jak z tym zadaniem poradzi sobie algorytm k średnich. Pierwszy krok to oczywiście wyznaczenie najlepszej wartości k .

```
[4]: # Listy w których będziemy zapisywać poszczególne wartości parametru k i
      # odpowiadających im wartości
      # metryki uśrednionej odległości pomiędzy punktami.
      k_values = []
      inertia_scores_vec = []

      # Następnie w pętli wywołujemy algorytm k średnich i zapisujemy wartości k oraz
      # intercji.
      # Tym razem wykorzystamy domyślne nastawy pracy algorytmu przyjęte w bibliotece
      # scikit-learn.
      for k in range(2,15):
          clusterer = KMeans(n_clusters=k)
          clusterer.fit(dataset)
          inertia_scores_vec.append(clusterer.inertia_)
          k_values.append(k)

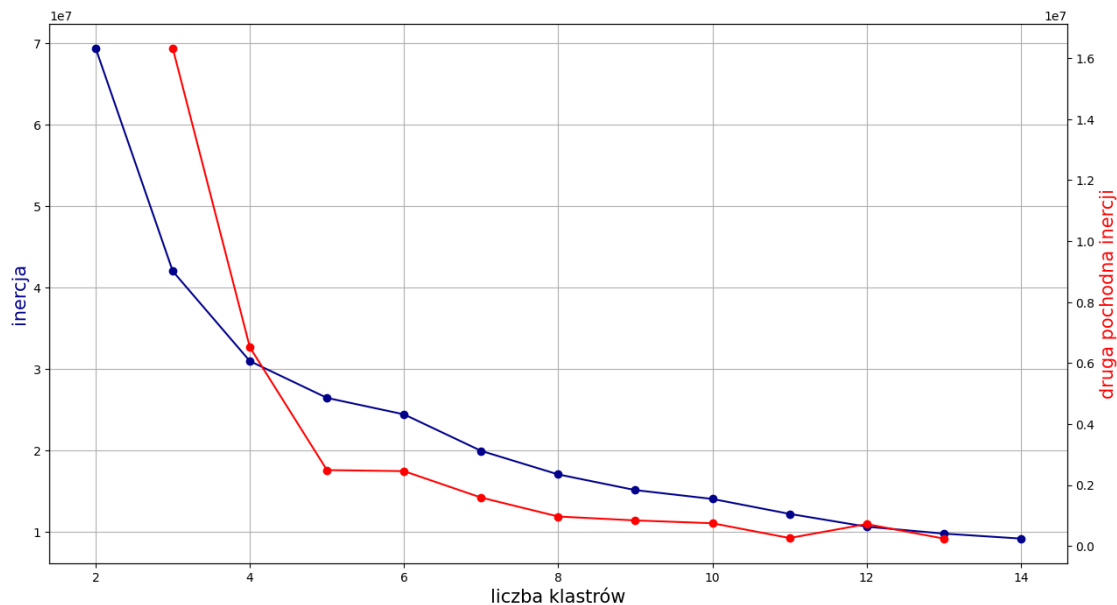
      # Na koniec rysujemy wykres inercji w funkcji liczby klastrów k.
      fig = plt.figure(figsize=(15,8))
```

```
plt.plot(k_values, inertia_scores_vec, marker='o', c='darkblue')
plt.grid()

# Na dodatkowej (prawej) osi y narysujemy drugą pochodną funkcji inercji, co
# pozwoli łatwiej odnaleźć punkt
# gdzie dynamika zmian inercji (przyspieszenie) jest możliwie mała. W
# najlepszym przypadku - będzie to minimum
# modułu drugiej pochodnej.
ax1 = plt.gca()
ax2 = ax1.twinx()
module_of_second_derivative = np.abs(np.diff(np.diff(inertia_scores_vec)))
ax2.plot(k_values[1:-1], module_of_second_derivative, marker='o', color='red')

# Opis osi
ax1.set_xlabel('liczba klastrów', fontsize=15)
ax1.set_ylabel('inercja', color='darkblue', fontsize=15)
ax2.set_ylabel('druga pochodna inercji', color='red', fontsize=15)
```

[4]: Text(0, 0.5, 'druga pochodna inercji')



Wyniki widoczne na wykresie inercji nie są jednoznaczne, konieczne będzie wypróbowanie kilku wartości parametru k . Minima drugiej pochodnej inercji widoczne są dla $k \in [7, 9, 12]$. Na początek, spróbujmy wypróbować wartość 7.

```
[5]: # Liczba klastrów, której się spodziewamy na podstawie wykresu inercji.
n_clusters = 7
```

```

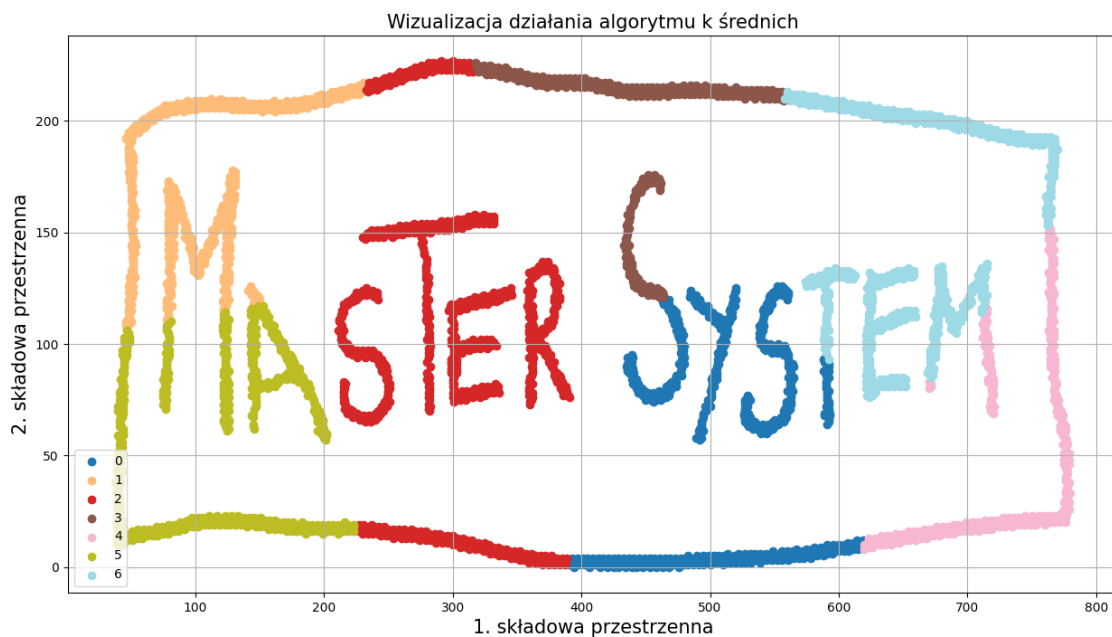
# Wykonanie algorytmu k średnich
clusterer = KMeans(n_clusters=n_clusters)
clusterer.fit(dataset)
k_means_predictions = clusterer.predict(dataset)

# Wizualizacja za pomocą wcześniej wprowadzonej już funkcji
↳ plot_decision_regions.
plt.figure(figsize=(15,8))
colors = cm.tab20(np.linspace(0, 1, len(np.unique(k_means_predictions))))
# Narysujemy wykres punktowy z rozdzieleniem danych na klasy.
for val,cv in zip(np.unique(k_means_predictions),colors):
    # Generujemy maskę za pomocą której wybierzemy punkty należące do każdej z
    ↳ klas.
    mask = k_means_predictions==val
    plt.scatter(dataset[mask,0],dataset[mask,1], label=val, color=cv)

# Oznaczenie osi i dodatkowe elementy wykresu.
plt.xlabel('1. składowa przestrzenna', fontsize=15)
plt.ylabel('2. składowa przestrzenna', fontsize=15)
plt.title("Wizualizacja działania algorytmu k średnich", fontsize=15)
plt.grid()
plt.legend(loc='lower left', fontsize=10)

```

[5]: <matplotlib.legend.Legend at 0x1fda3859010>



Ramka okalająca napis jest podzielona pomiędzy wiele klastrów, klastry zawierające wiele liter

zawierają nie tylko całe litery, ale i fragmenty liter sąsiednich. W ramach dalszych zadań konieczne będzie sprawdzenie wyższych wartości parametru k .

1.3 Algorytm hierarchiczny

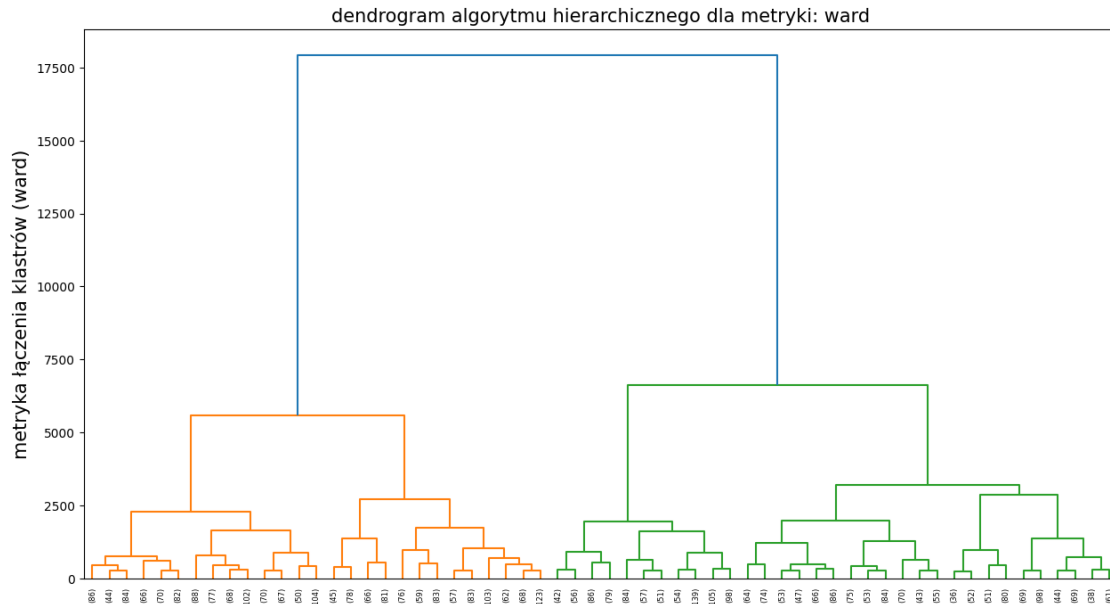
Warto wypróbować także algorytm hierarchiczny. Podobnie jak dla algorytmu k średnich tu także konieczna będzie decyzja odnośnie tego, jaką liczbę klastrow wybrać. Rozpocząć zatem należy od narysowania dendrogramu. Warto sprawdzić, jak on wygląda dla różnych metryk łączenia punktów w klastry. Zaczniemy od sprawdzenia, jak algorytm działa, gdy wybrana jest metryka Warda.

```
[6]: # Dostępne metryki łączenia klastrow:
#     'ward'      - metryka minimalizująca wariancję łączonych klastrow
#     'average'   - maksymalizuje średni dystans pomiędzy punktami
#                  w różnych klastrach
#     'complete' - minimalizuje maksymalną odległość
#                  między punktami w klastrach
#     'single'    - maksymalizuje najmniejszą odległość
#                  między punktami w klastrach

# Wybieramy metrykę według której łączymy poszczególne punkty danych
linkage_method = 'ward'

# Rysujemy dendrogram
plt.figure(figsize=(15,8))
dend = sch.dendrogram(sch.linkage(dataset, method=linkage_method),
    ↳ truncate_mode='lastp', p=60)
plt.ylabel(f'metryka łączenia klastrow ({linkage_method})', fontsize=15)
plt.title(f'dendrogram algorytmu hierarchicznego dla metryki: ↳
    ↳ {linkage_method}', fontsize=15)
```

```
[6]: Text(0.5, 1.0, 'dendrogram algorytmu hierarchicznego dla metryki: ward')
```



Oceniając wizualnie dendrogram - potencjalnie liczba klastrow może wynosić 2, albo 4. Spróbujmy wypróbować wartość 4.

```
[7]: # Liczba klastrow, której się spodziewamy na podstawie wyglądu dendrogramu.
n_clusters = 4

# Wykonanie algorytmu hierarchicznego:
clusterer = AgglomerativeClustering(n_clusters=n_clusters,
    ↪linkage='ward')
clusterer.fit(dataset)

# Zapisujemy sobie wynik przypisania danych do klastrow.
agclust_predictions = clusterer.labels_

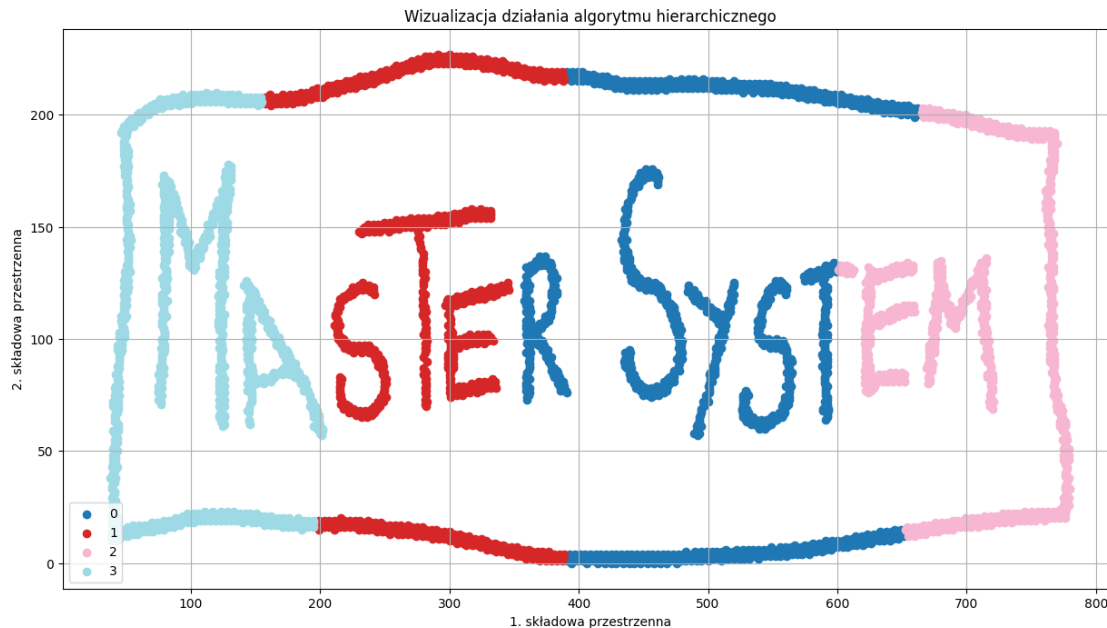
# Wizualizacja za pomocą wcześniej wprowadzonej już funkcji
    ↪plot_decision_regions.
plt.figure(figsize=(15,8))
colors = cm.tab20(np.linspace(0, 1, len(np.unique(agclust_predictions))))
# Narysujemy wykres punktowy z rozdzieleniem danych na klasy.
for val,cv in zip(np.unique(agclust_predictions),colors):
    # Generujemy maskę za pomocą której wybierzemy punkty należące do każdej z
    ↪klas.
    mask = agclust_predictions==val
    plt.scatter(dataset[mask,0],dataset[mask,1], label=val, color=cv)

# Oznaczenia osi i pozostałe elementy wykresu.
plt.xlabel('1. składowa przestrzenna')
```



```
plt.ylabel('2. składowa przestrzena')
plt.title("Wizualizacja działania algorytmu hierarchicznego")
plt.grid()
plt.legend(loc = 'lower left')
```

[7]: <matplotlib.legend.Legend at 0x1fda33d0830>



Widoczny jest dość równomierny podział loga, jednak także tutaj należy sprawdzić, czy możliwy jest lepszy efekt klasyfikacji, który pozwoli na oddzielenie od siebie liter w logi i otaczającej je ramki.

1.4 Algorytmy analizujące gęstość punktów

Konieczność wyboru liczby klastrow jeszcze przed dokonaniem klasyfikacji bywa wymogiem problematycznym. Zmusza on to prowadzenia dodatkowych analizy mających na celu ustalenie optymalnej liczby klas. Z tego powodu opracowywane są także takie techniki, które nie mają takiego wymogu. Przykładem algorytmu tego typu jest algorytm DBSCAN. W uproszczeniu - jego zasada działania opiera się na analizie gęstości punktów o wydzielaniu do osobnych klas skupisk punktów o wysokiej gęstości w porównaniu do gęstości punktów w przestrzeni, która te skupiska otacza. Stąd, jeżeli grupy punktów są dobrze od siebie odseparowane, to możliwe jest wykorzystanie tego typu algorytmu do oddzielenia od siebie skupisk punktów bez konieczności uprzedniego liczenia, np. ile znaków zawiera segmentowany napis.

Należy jednak pamiętać, że nawet taki algorytm, jak DBSCAN wymaga jednak specyficznych nastaw. W jego przypadku konieczne jest zwykle zdefiniowanie dwóch kluczowych wartości:

minimalnej liczby punktów, która okala tzw. punkty rdzeniowe (ang. *core samples*), próbki te stanowią “zaczątki” klas generowanych przez algorytm DBSCAN (oznaczana w scikit-learn jako `min_samples`),

maksymalnej odległości, w promieniu której (licząc od punktów rdzeniowych) poszukiwane są punkty okalające punkty rdzeniowe (oznaczana w scikit-learn jako eps).

Oznacza to, że algorytm tworzy próbkę rdzeniową, jeżeli w promieniu o wartości eps od wybranego punktu w zbiorze danych znajduje się przynajmniej min_samples innych punktów. Poszukiwanie takich próbek rdzeniowych jest kontynuowane do momentu aż nie będzie możliwe wyróżnienie nowych próbek rdzeniowych. Gdy już nie będzie możliwe znalezienie próbek rdzeniowych, to jeżeli jakiś punkt znajduje się w odległości mniejszej lub równej eps od punktu będącego próbką rdzeniową jakiegoś klastra, to przypisuje się go do właśnie tego klastra. Punkty, dla których nie udało się wykonać nawet takiego przypisania, pozostają nieprzypisane do żadnej klasy.

Umiejętny dobór wartości min_samples i eps pozwala na modyfikowanie zachowania się algorytmu, ich dobór często jest silnie powiązany z klasyfikowanym zbiorem danych, dlatego też w tym przykładzie, dobierzemy je eksperymentalnie, oceniając efekty segmentacji tekstu.

Dodatkowe informacje i źródła na temat algorytmu DBSCAN są możliwe do znalezienia w [dokumentacji](#) biblioteki scikit-learn.

```
[8]: # Nastawy klasyfikacji nienadzorowanej algorytmem DBSCAN
min_samples = 20
eps = 60

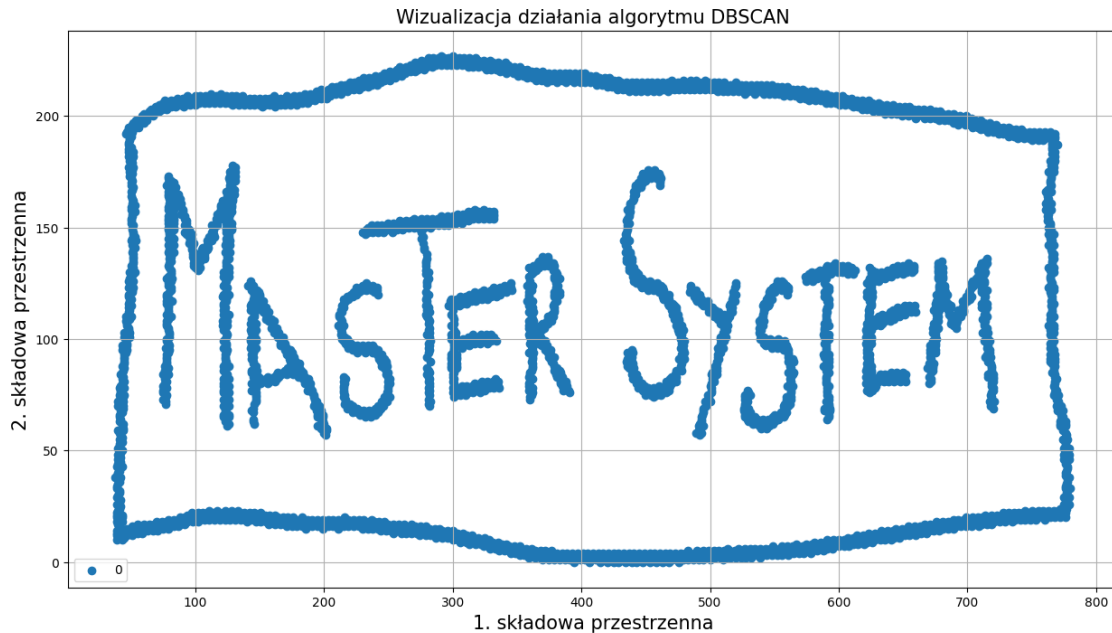
# Wykonanie algorytmu DBSCAN
clustering = DBSCAN(min_samples=min_samples,eps=eps).fit(dataset)
dens_predictions = clustering.labels_

# Wizualizacja za pomocą wcześniej wprowadzonej już funkcji
↳plot_decision_regions.
plt.figure(figsize=(15,8))
colors = cm.tab20(np.linspace(0, 1, len(np.unique(dens_predictions))))
# Narysujemy wykres punktowy z rozdzieleniem danych na klasy.
for val,cv in zip(np.unique(dens_predictions),colors):
    # Generujemy maskę za pomocą której wybierzemy punkty należące do każdej z
    ↳klas.
    mask = dens_predictions==val
    # Dla tak wszystkich punktów wybranych za pomocą odpowiadającej danej
    ↳klasy maski rysujemy wykres punktowy.
    if val == -1:
        c = 'black'; marker = 'x'
    else:
        c = cv; marker = 'o'
    plt.scatter(dataset[mask,0],dataset[mask,1], label=val, color=c,
    ↳marker=marker)

# Oznaczenia osi i pozostałe elementy wykresu.
plt.xlabel('1. składowa przestrzenna', fontsize=15)
plt.ylabel('2. składowa przestrzenna', fontsize=15)
plt.title("Wizualizacja działania algorytmu DBSCAN", fontsize=15)
plt.grid()
```

```
plt.legend(loc='lower left')
```

```
[8]: <matplotlib.legend.Legend at 0x1fda34396a0>
```



Efektem wykonania algorytmu DBSCAN z obecnymi wartościami nastaw jest zaklasyfikowanie całego zbioru punktów do pojedynczego klastra. Oznacza to, że prawdopodobnie parametr `eps` jest za duży i pozwala on na “przeskoczenie” poszukiwania sąsiednich punktów przez przerwy oddzielające od siebie poszczególne znaki loga. Warto zatem rozpocząć poszukiwania lepiej sprawdzających się nastaw od zmniejszenia tego parametru.

2 Zadania do wykonania

2.1 Zadanie 1.

Obecne zbiory nastaw algorytmów klasyfikacji nienadzorowanej nie okazały się zbyt skuteczne w realizacji zadania segmentacji znaków w przykładowym, ręcznie narysowanym logu. Poszukaj lepszych nastaw algorytmów i postaraj się możliwie najdokładniej rozdzielić litery i ramkę loga poprzez zaklasyfikowanie ich do oddzielnych klas za pomocą badanych w przykładzie algorytmów.

Czy wszystkie algorytmy poradziły sobie równie dobrze w zadaniu segmentacji pisma? Jakie problemy z segmentacją pisma udało Ci się zaobserwować?

Jeśli jakiś algorytm nie poradził sobie w zadowalającym stopniu z tym zadaniem, dlaczego mogło się tak stać? Podobnie w przypadku algorytmów, którym udało się dokonać segmentacji - dlaczego akurat taki dobór nastaw okazał się najbardziej skuteczny?

Czy wygodniejsze do zadania segmentacji pisma są algorytmy wymagające znajomości liczby klas, czy te bazujące na gęstości punktów? Czy może istnieją Twoim zdaniem zastosowania, w

których jedno z nich spisują się lepiej i na odwrót?

2.2 Zadanie 2.

Zamień teraz plik wejściowy na plik “wzor_matematyczny.png”. Uruchom ponownie (pozostawiając nastawy algorytmów, wprowadzone przez Ciebie w zadaniu 1.) wszystkie algorytmy klasyfikacji nienadzorowanej. Następnie, jeżeli któryś z algorytmów wymaga poprawienia nastaw, zmień je tak, aby znowu uzyskać rozdzielenie liter na osobne klasy.

Czy był taki algorytm, który nie wymagał zmiany nastaw aby poprawnie dokonać segmentacji pisma?

Jakie nowe problemy pojawiły się w przypadku segmentacji symboli matematycznych? Jakie poprawki lub jakie dodatkowe przetwarzanie należy jeszcze dodać, jeżeli efekt segmentacji z ćwiczenia miałyby posłużyć np. do segmentacji znaków na potrzeby systemu rozpoznawania liter (OCR)?

Jakie zalety i wady ma Twoim zdaniem algorytm DBSCAN, jeśli porównać go z algorytmem k-średnich i hierarchicznym?

2.3 Rozwiązanie zadań

2.3.1 Zadanie 1 - logo.png

Najpierw sprawdzam kilka sensowniejszych nastaw dla loga. Używam tego samego przetworzenia obrazu co wyżej, tylko zamykam je w małej funkcji, żeby potem łatwo przełączyć plik wejściowy.

```
[9]: # rozwiązanie_p3
# Funkcje pomocnicze do zadań
def prepare_dataset(file_name, step=4):
    image = Image.open(file_name)
    image_bw = np.array(image.convert('1'))
    coords = np.where(image_bw == 0)
    dataset = np.vstack([coords[1], np.max(coords[0]) - coords[0]]).T
    return dataset[:, :step, :]

def plot_clusters(dataset, labels, title):
    plt.figure(figsize=(15, 8))
    values = np.unique(labels)
    colors = cm.tab20(np.linspace(0, 1, max(len(values), 2)))
    for val, cv in zip(values, colors):
        mask = labels == val
        if val == -1:
            plt.scatter(dataset[mask, 0], dataset[mask, 1], label='szum',
                color='black', marker='x', s=25)
        else:
            plt.scatter(dataset[mask, 0], dataset[mask, 1], label=val,
                color=cv, s=25)
    plt.xlabel('1. składowa przestrzenna', fontsize=15)
```

```

plt.ylabel('2. składowa przestrzenna', fontsize=15)
plt.title(title, fontsize=15)
plt.grid()
plt.legend(loc='lower left', fontsize=9)

def cluster_count(labels):
    return len(set(labels)) - (1 if -1 in labels else 0)

def noise_ratio(labels):
    return float(np.mean(labels == -1))

```

```

[10]: # rozwiązanie_p3
# Dane dla logo
dataset_logo = prepare_dataset('logo.png')
print('Liczba punktów po przerzedzeniu:', dataset_logo.shape[0])

# Ustawienia dobrane eksperymentalnie dla logo
logo_kmeans_k = 18
logo_aggl_k = 18
logo_aggl_linkage = 'average'
logo_dbscan_min_samples = 3
logo_dbscan_eps = 8

# KMeans
kmeans_logo = KMeans(n_clusters=logo_kmeans_k, random_state=42, n_init=10)
kmeans_logo_labels = kmeans_logo.fit_predict(dataset_logo)
plot_clusters(dataset_logo, kmeans_logo_labels, f'KMeans dla logo,
↳k={logo_kmeans_k}')

# Algorytm hierarchiczny
aggl_logo = AgglomerativeClustering(n_clusters=logo_aggl_k,
↳linkage=logo_aggl_linkage)
aggl_logo_labels = aggl_logo.fit_predict(dataset_logo)
plot_clusters(dataset_logo, aggl_logo_labels, f'Algorytm hierarchiczny dla
↳logo, linkage={logo_aggl_linkage}, k={logo_aggl_k}')

# DBSCAN
dbscan_logo = DBSCAN(min_samples=logo_dbscan_min_samples, eps=logo_dbscan_eps)
dbscan_logo_labels = dbscan_logo.fit_predict(dataset_logo)
plot_clusters(dataset_logo, dbscan_logo_labels, f'DBSCAN dla logo,
↳min_samples={logo_dbscan_min_samples}, eps={logo_dbscan_eps}')

print('KMeans - liczba klastrów:', cluster_count(kmeans_logo_labels))
print('Hierarchiczny - liczba klastrów:', cluster_count(aggl_logo_labels))

```

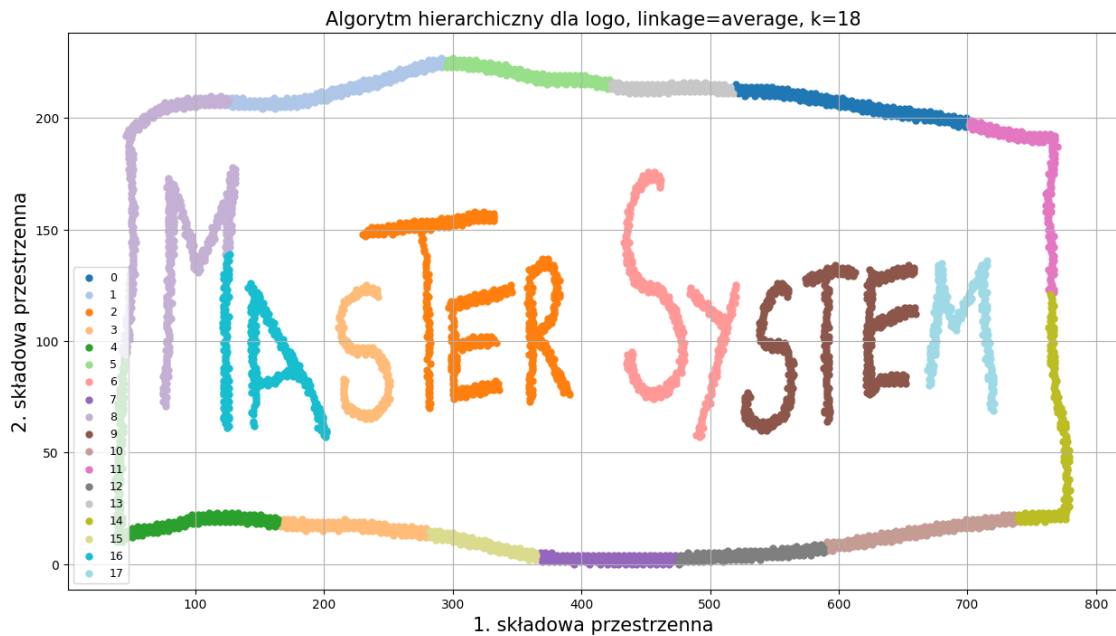
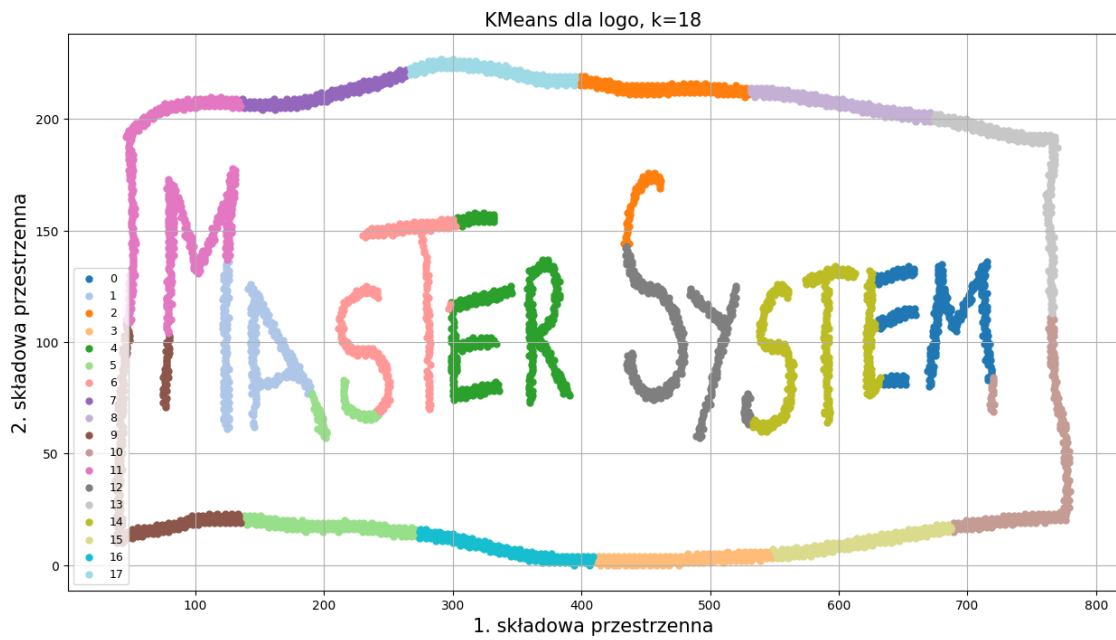
```
print('DBSCAN - liczba klastrow:', cluster_count(dbscan_logo_labels), '| szum:
↪', round(noise_ratio(dbscan_logo_labels), 4))
```

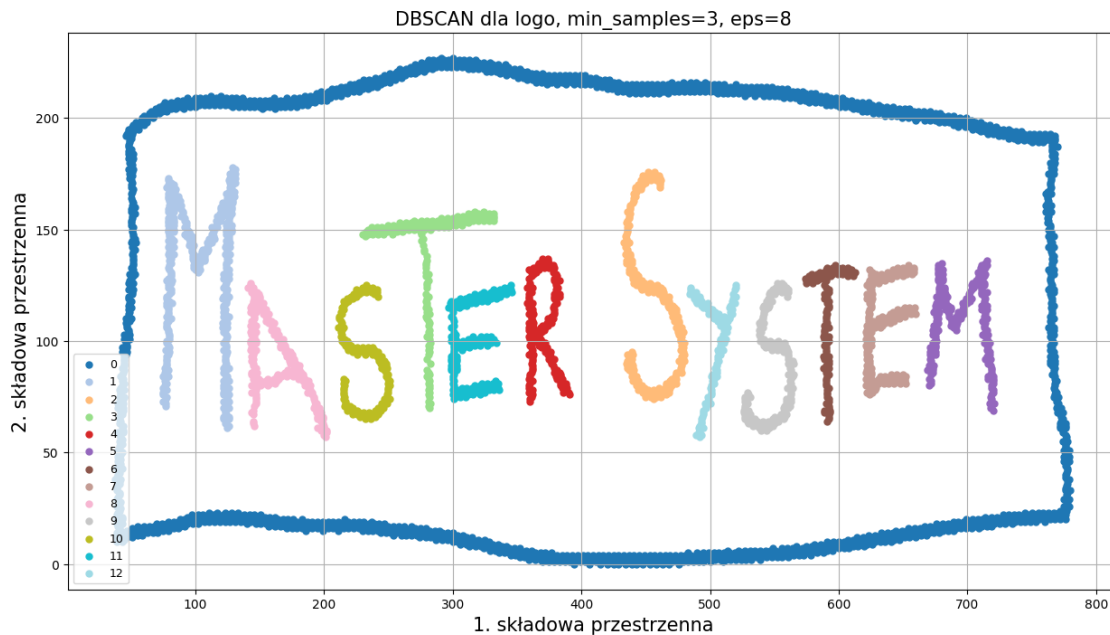
Liczba punktów po przerzedzeniu: 4261

KMeans - liczba klastrow: 18

Hierarchiczny - liczba klastrow: 18

DBSCAN - liczba klastrow: 13 | szum: 0.0





Wnioski po uruchomieniu:

- Najlepiej wyszedł DBSCAN. Dla $\text{eps}=8$ i $\text{min_samples}=3$ dostałem 13 klastrow, czyli praktycznie ramkę + 12 liter z napisu MASTER SYSTEM. To jest najbliższe tego, czego oczekujemy w segmentacji.
- KMeans nawet przy większym $k=18$ nadal tnie obiekty według położenia w przestrzeni. Ramka jest pokrojona na fragmenty, a część liter też dostaje sztuczne podziały. No działa, ale trochę na siłę.
- Algorytm hierarchiczny z $\text{linkage}=\text{'average'}$ i $k=18$ wygląda podobnie: potrafi rozdzielać obszary, ale nie rozumie, że cała ramka jest jednym znakiem/obiektem.
- Problem wynika głównie z kształtu danych. KMeans i hierarchiczny muszą dostać liczbę klastrow, a do tego mocno patrzą na odległości globalne. DBSCAN patrzy lokalnie na gęstość i połączenia między punktami, więc dla oddzielonych kresek jest dużo wygodniejszy.
- Dla takich obrazków, gdzie znaki są osobnymi ciągłymi plamami, algorytm gęstościowy jest praktyczniejszy. Gdyby jednak znaki się stykały albo miały bardzo różną grubość kreski, DBSCAN też mógłby się pogubić i wtedy trzeba by dodać przetwarzanie obrazu przed klasteryzacją.

2.3.2 Zadanie 2 - wzor_matematyczny.png

Teraz zmieniam wejście na `wzor_matematyczny.png`. Najpierw puszczam algorytmy na ustawieniach z zadania 1, a potem poprawiam tylko te, które ewidentnie wymagają zmiany.

```
[11]: # rozwiazanie_p3
      # Dane dla wzoru matematycznego
```

```

dataset_math = prepare_dataset('wzor_matematyczny.png')
print('Liczba punktów po przerzedzeniu:', dataset_math.shape[0])

# Uruchomienie z nastawami z zadania 1
kmeans_math_old_labels = KMeans(n_clusters=logo_kmeans_k, random_state=42,
    ↪n_init=10).fit_predict(dataset_math)
aggl_math_old_labels = AgglomerativeClustering(n_clusters=logo_aggl_k,
    ↪linkage=logo_aggl_linkage).fit_predict(dataset_math)
dbscan_math_old_labels = DBSCAN(min_samples=logo_dbscan_min_samples,
    ↪eps=logo_dbscan_eps).fit_predict(dataset_math)

plot_clusters(dataset_math, kmeans_math_old_labels, f'KMeans dla wzoru, nastawy
    ↪z zad. 1, k={logo_kmeans_k}')
plot_clusters(dataset_math, aggl_math_old_labels, f'Hierarchiczny dla wzoru,
    ↪nastawy z zad. 1, k={logo_aggl_k}')
plot_clusters(dataset_math, dbscan_math_old_labels, f'DBSCAN dla wzoru, nastawy
    ↪z zad. 1, eps={logo_dbscan_eps}')

print('KMeans - liczba klastrów:', cluster_count(kmeans_math_old_labels))
print('Hierarchiczny - liczba klastrów:', cluster_count(aggl_math_old_labels))
print('DBSCAN - liczba klastrów:', cluster_count(dbscan_math_old_labels), '|
    ↪szum:', round(noise_ratio(dbscan_math_old_labels), 4))

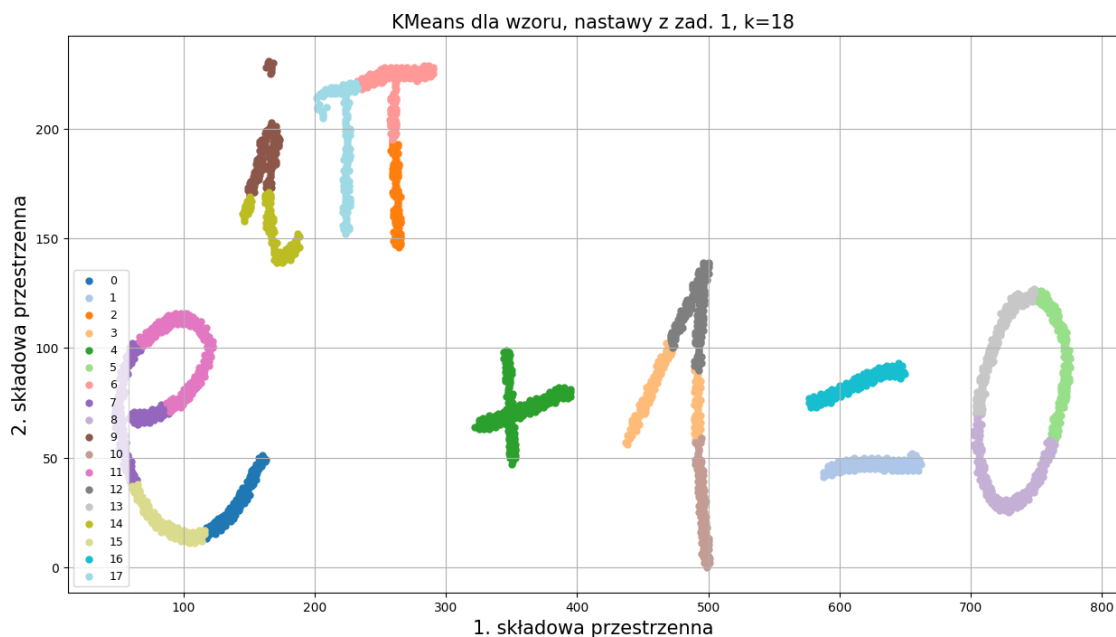
```

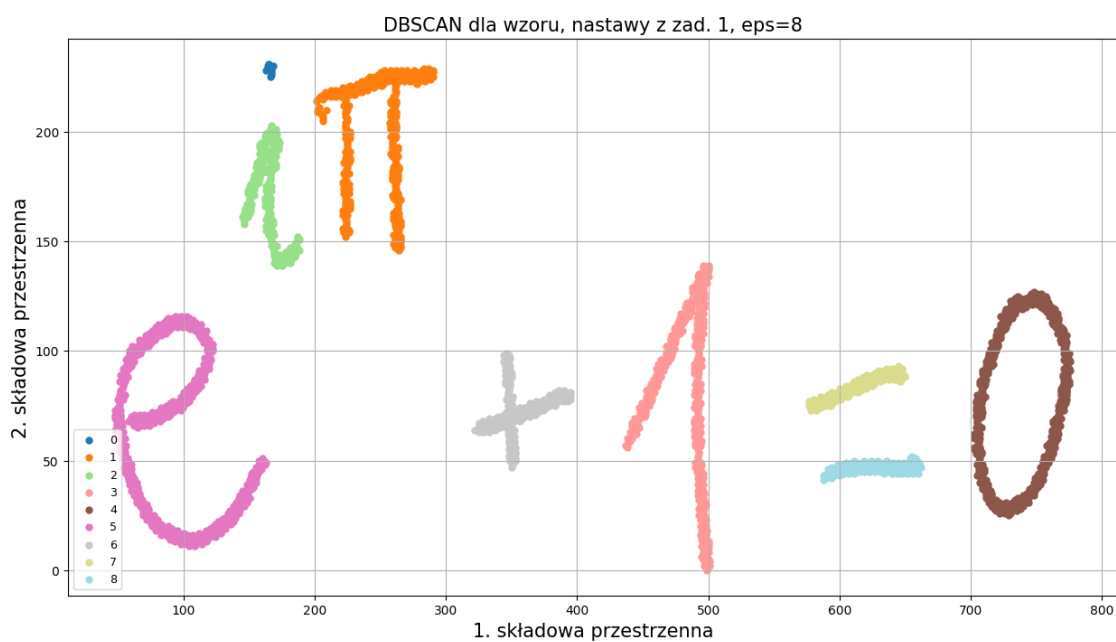
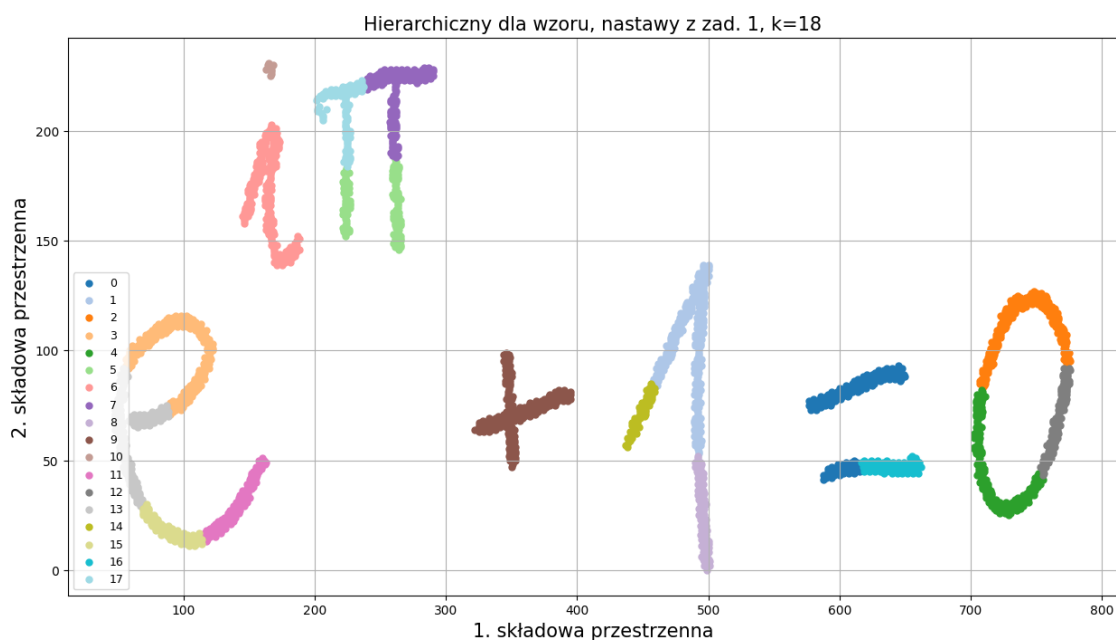
Liczba punktów po przerzedzeniu: 2199

KMeans - liczba klastrów: 18

Hierarchiczny - liczba klastrów: 18

DBSCAN - liczba klastrów: 9 | szum: 0.0





Po pierwszym uruchomieniu widać, że nastawy z loga nie pasują do wszystkich metod. KMeans i hierarchiczny mają za duże k, więc rozbijają znaki na niepotrzebne kawałki. DBSCAN nie wymagał poprawki: dalej rozdziela odseparowane kreski, choć traktuje kropkę nad i i dwie kreski znaku = jako osobne klastry.

```
[12]: # rozwiazanie_p3
# Poprawione ustawienia dla wzoru matematycznego
math_kmeans_k = 9
math_aggl_k = 9
math_aggl_linkage = 'average'
math_dbscan_min_samples = logo_dbscan_min_samples
math_dbscan_eps = logo_dbscan_eps

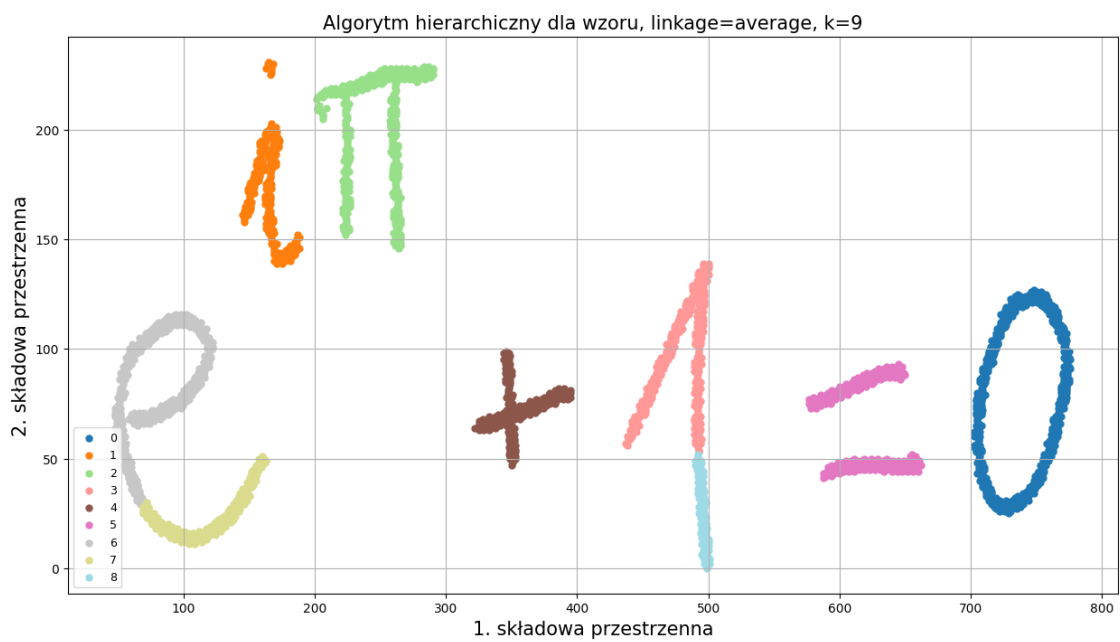
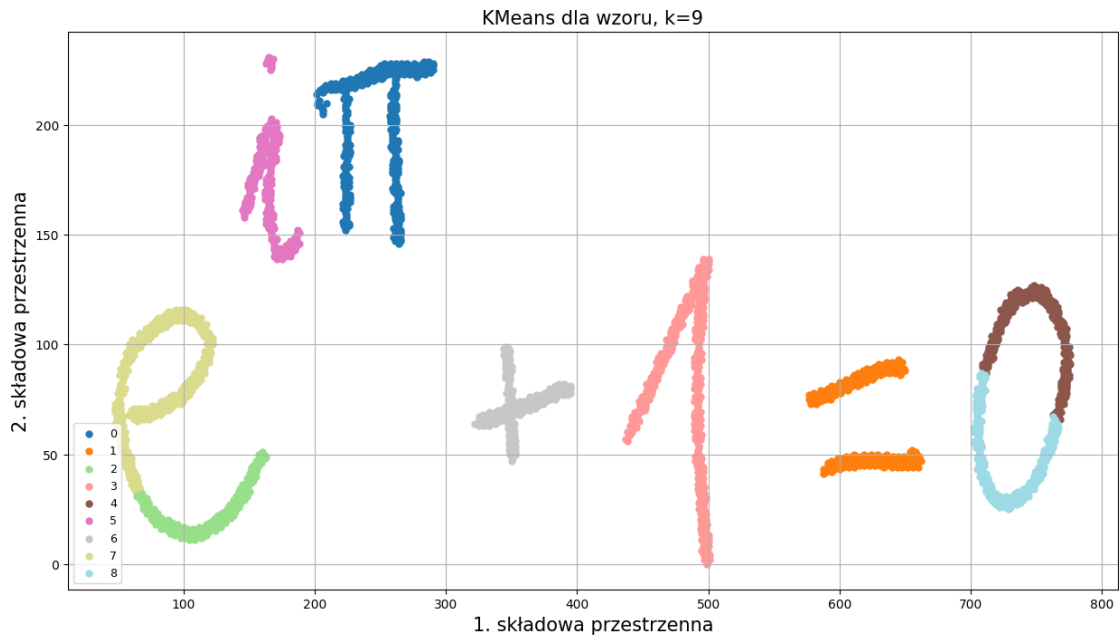
kmeans_math = KMeans(n_clusters=math_kmeans_k, random_state=42, n_init=10)
kmeans_math_labels = kmeans_math.fit_predict(dataset_math)
plot_clusters(dataset_math, kmeans_math_labels, f'KMeans dla wzoru,
↳ k={math_kmeans_k}')

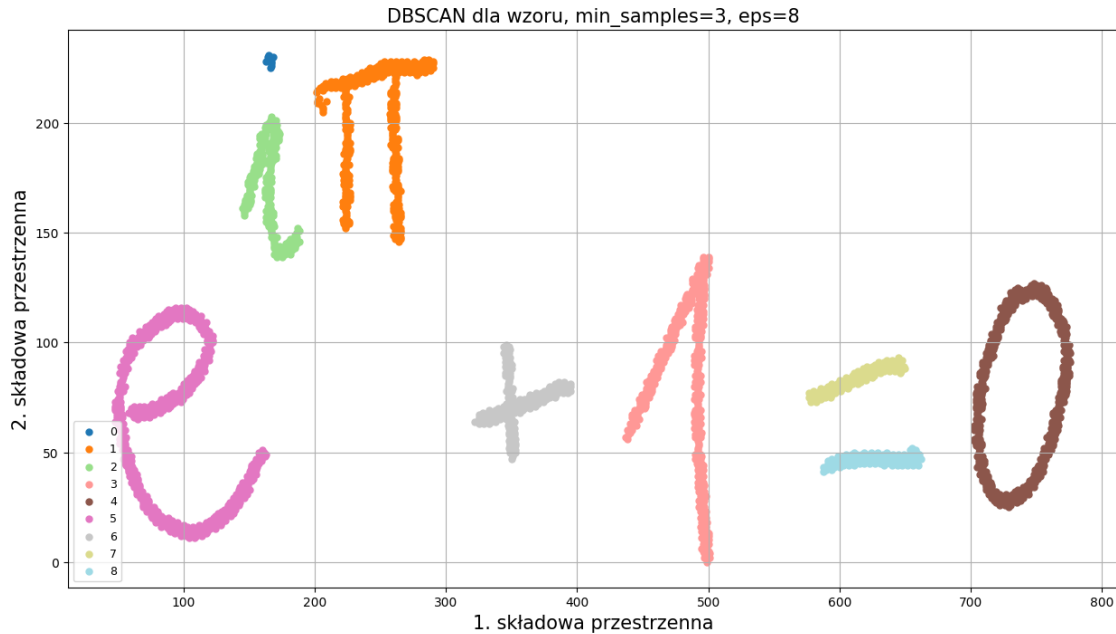
aggl_math = AgglomerativeClustering(n_clusters=math_aggl_k,
↳ linkage=math_aggl_linkage)
aggl_math_labels = aggl_math.fit_predict(dataset_math)
plot_clusters(dataset_math, aggl_math_labels, f'Algorytm hierarchiczny dla
↳ wzoru, linkage={math_aggl_linkage}, k={math_aggl_k}')

dbscan_math = DBSCAN(min_samples=math_dbscan_min_samples, eps=math_dbscan_eps)
dbscan_math_labels = dbscan_math.fit_predict(dataset_math)
plot_clusters(dataset_math, dbscan_math_labels, f'DBSCAN dla wzoru,
↳ min_samples={math_dbscan_min_samples}, eps={math_dbscan_eps}')

print('KMeans - liczba klastrów:', cluster_count(kmeans_math_labels))
print('Hierarchiczny - liczba klastrów:', cluster_count(aggl_math_labels))
print('DBSCAN - liczba klastrów:', cluster_count(dbscan_math_labels), '| szum:
↳ ', round(noise_ratio(dbscan_math_labels), 4))
```

```
KMeans - liczba klastrów: 9
Hierarchiczny - liczba klastrów: 9
DBSCAN - liczba klastrów: 9 | szum: 0.0
```





Wnioski po uruchomieniu:

- Dla wzoru matematycznego bez zmiany nastaw najlepiej poradził sobie DBSCAN. Parametry z loga nadal dały sensowny podział na 9 klastrów.
- KMeans i algorytm hierarchiczny trzeba było zmniejszyć do około $k=9$, bo przy $k=18$ dzieliły pojedyncze symbole na fragmenty. Po zmianie jest lepiej, ale dalej nie jest to idealne rozpoznawanie znaków, tylko segmentacja punktów.
- Nowy problem jest taki, że nie każdy znak jest jedną ciągłą kreską. Kropka nad i jest osobnym elementem, a znak $=$ składa się z dwóch oddzielnych kresek. DBSCAN naturalnie rozdziela takie części, więc do OCR trzeba by potem dodać etap łączenia bliskich komponentów w jeden symbol.
- Przydałoby się też wstępne czyszczenie obrazu, usuwanie szumu, normalizacja grubości kreski i analiza położenia elementów względem siebie, np. żeby wiedzieć, że kropka należy do i , a dwie równoległe kreski tworzą $=$.
- DBSCAN ma tę zaletę, że nie trzeba z góry znać liczby klastrów i bardzo dobrze łapie oddzielone komponenty. Wadą jest czułość na ϵ oraz to, że rozdziela znaki wieloczęściowe. KMeans i hierarchiczny są prostsze do kontrolowania przez k , ale przy piśmie ręcznym często tną obraz dość sztucznie.

